Claims Management App: Quality Engineering

System Restoration & Robust Testing Framework Strategy

EXECUTIVE OVERVIEW

Project Mission

Transition from a broken "hollow skeleton" codebase to a testable, production-ready quality engineering framework by restoring core backend logic and UI functionality.

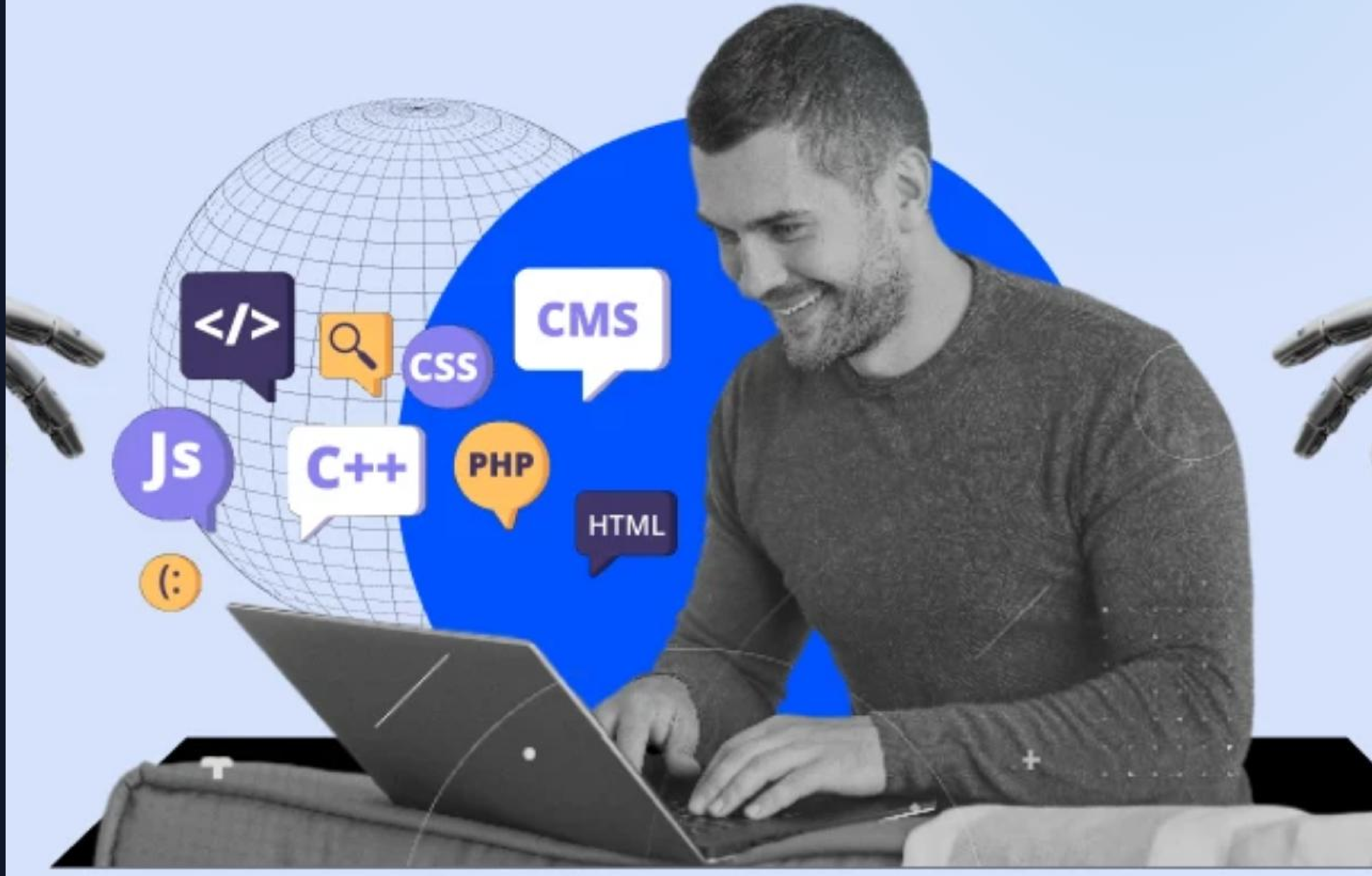
Core Strategy

Prioritizing **"Quality over Quantity"**. We focused on foundational stability and high-confidence E2E coverage rather than superficial feature additions.

THE INITIAL CHALLENGE

The project arrived as a non-functional skeleton with critical gaps:

- **Missing Files:** Build configs and service models.
- **Broken Logic:** Core backend services were non-responsive.
- **Non-Buildable:** Incomplete Dockerfiles and POM files.



MULTI-TIER TESTING STRATEGY

Unit Tests

Utilizing **JUnit 5** to validate API boundaries and complex business logic at the source level.

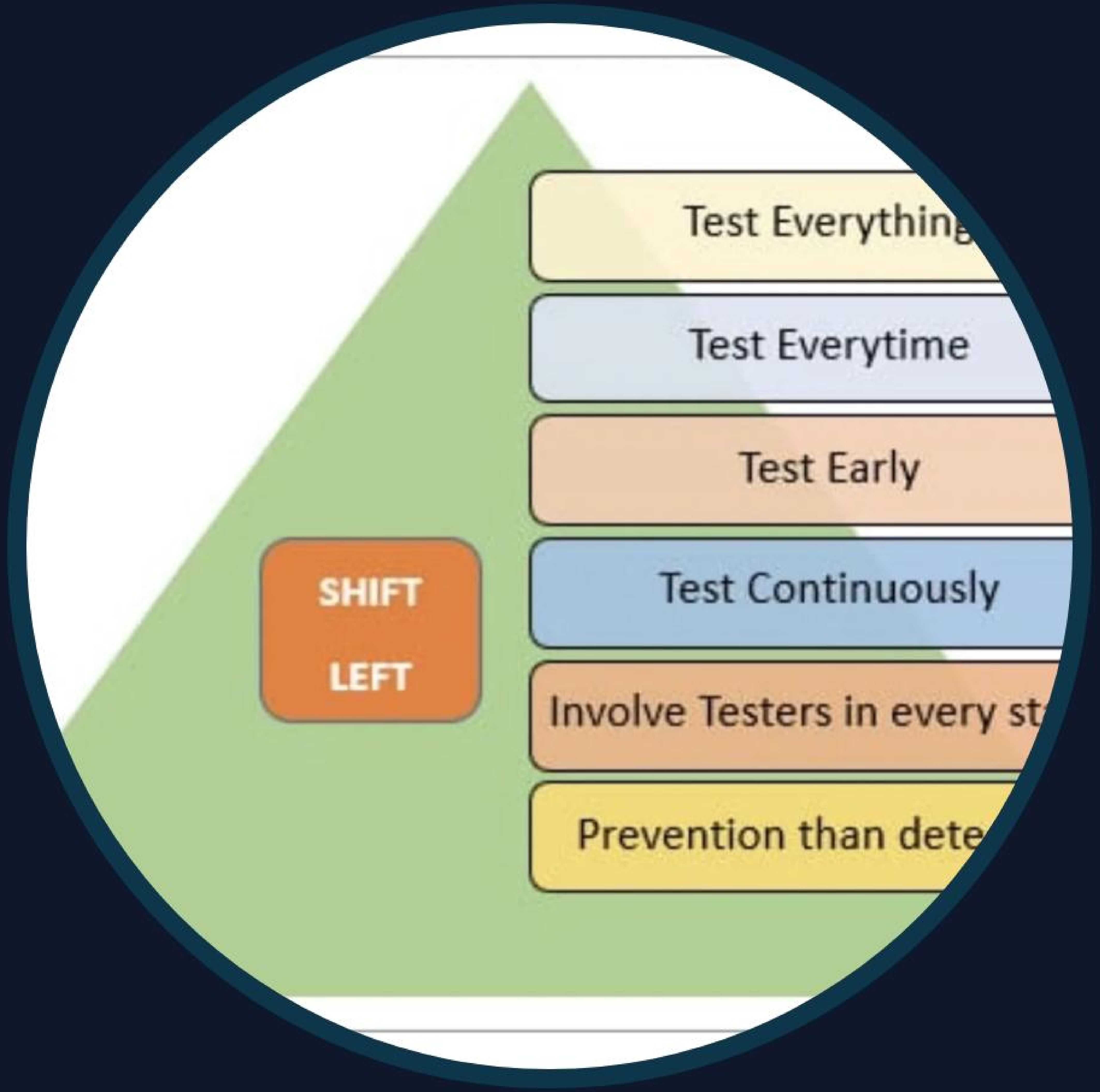
Component Tests

Skeleton validation using the **Angular Testing Library** to ensure UI elements render as expected.

E2E Tests

Full user journey verification using **Playwright** against a real, functional interface.

SHIFT-LEFT ENGINEERING PHILOSOPHY



The Rationale

By implementing a layered approach, we ensure that defects are caught during development (Shift Left) rather than at deployment. This provides high-confidence integration coverage and minimizes technical debt accumulation.

SCALABILITY WITH POM IMPLEMENTATION

- **Centrally Managed Locators:** INTERACTIONS and selectors are isolated in `ClaimFormPage.ts` for easier maintenance.
- **Semantic Locators:** A strategic shift from fragile XPaths to robust `getByRole` and `getByLabel` selectors for accessible-first testing.
- **Single Responsibility Principle:** Decoupling test logic from UI structure ensures that UI updates don't break the entire suite.

HYBRID FRONTEND STRATEGY

Continuity Under Construction

To ensure the test suite provided immediate value while the Angular app was being built, we maintained the Angular structure but provided a **functional HTML fallback** for E2E verification.

DESIGN RATIONALE: INTEGRITY OVER ILLUSION

0

MOCKED DOM TESTS

Removing "Fake" Validation

Initial tests mocked the HTML DOM inside test files, providing false confidence. We refactored the suite to interact with **real UI elements**, transforming a "passing but broken" suite into a meaningful verification tool.

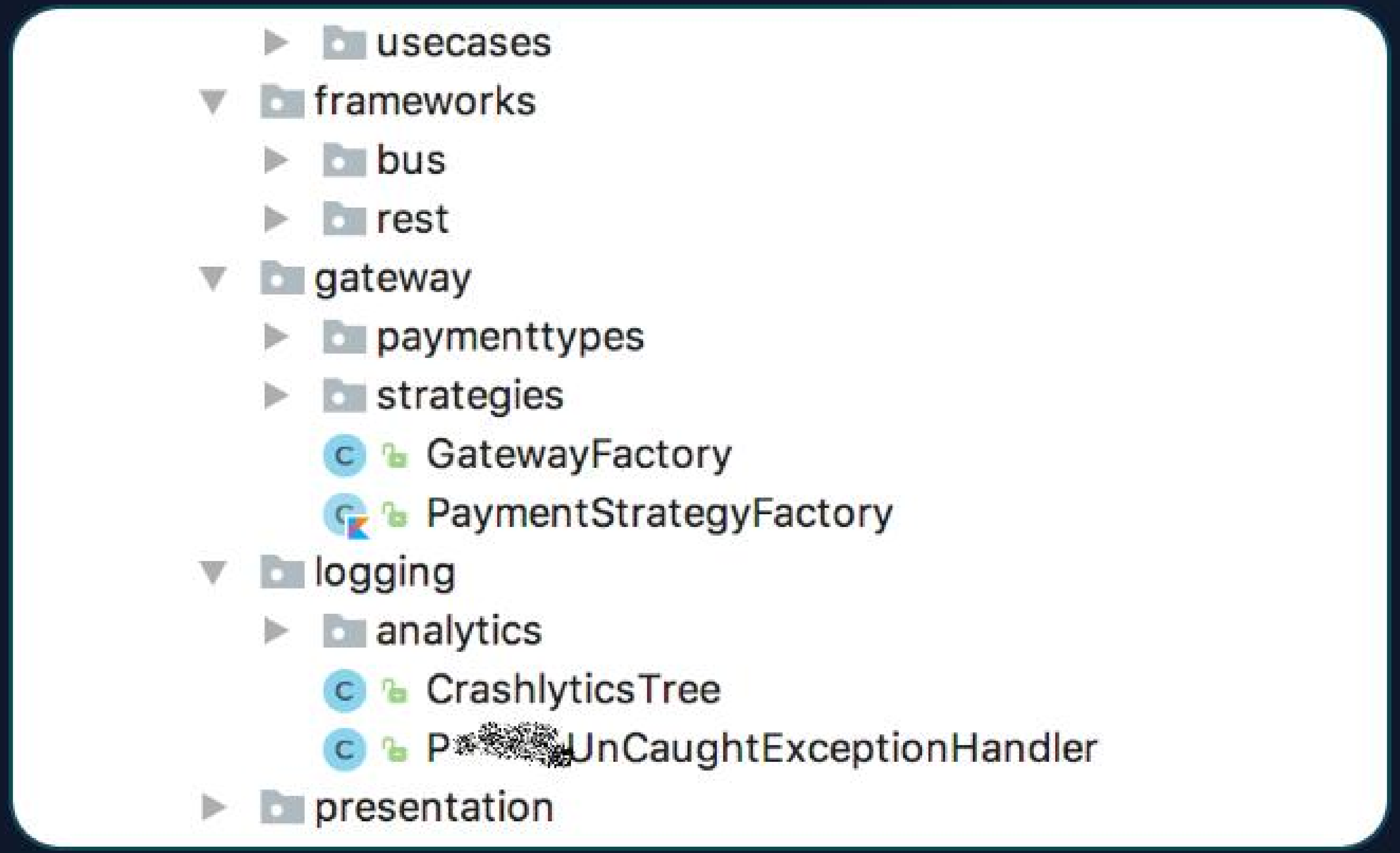
INFRASTRUCTURE RESTORATION

Docker Icon

Functional Dockerfiles for isolated builds.

OpenAPI Logo

OpenAPI Contracts defining the API spec.



Full Maven/POM restoration for building.

TECHNICAL TRADE-OFF ANALYSIS

Feature Decision	Trade-off / Rationale
Infrastructure vs. Full CI/CD	Restored "engine" (testability) over "highway" (pipelines) due to skeleton state.
Mocking Strategy	Mocked Backend APIs in E2E temporarily for environment stability and fast feedback.
Styling Frameworks	Stuck to Angular defaults/Plain HTML to minimize dependencies and focus on integrity.

FINAL DELIVERABLES & STATUS

- ☑ **Git Repository:** Clean, meaningful history showing restoration steps.
- ☑ **Working Platform:** `docker-compose.yml` for Backend & Frontend.
- ☑ **OpenAPI Specs:** `openapi.yaml` defining the API contract.
- ☑ **AI Working Journal:** Strategic reasoning in `AI_JOURNAL.md`.

CONCLUSION & FUTURE ROADMAP

Current

Restored Skeleton & Robust QE framework.

Next Phase

Kafka Integration (using Testcontainers).

Future

WebSocket State Verification in E2E.

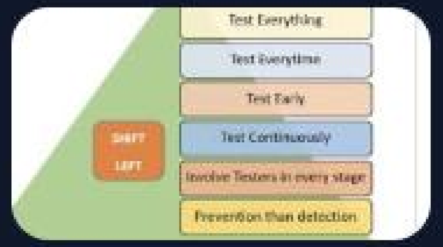
Scale

Visual Regression & UI Stability testing.

IMAGE SOURCES

https://s3.amazonaws.com/mobileappdaily/mad/uploads/header_image_mad_img_1752658932.webp

Source: www.mobileappdaily.com



https://codimite.ai/wp-content/uploads/Img02.webp

Source: codimite.ai

https://www.edureka.co/blog/wp-content/uploads/2025/02/angular-architecture-1.webp

Source: www.edureka.co

---

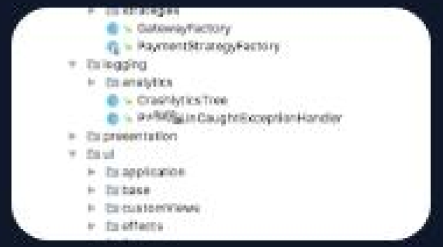
https://www.docker.com/app/uploads/2026/02/Docker_Mark_660x400.png

Source: www.docker.com

---

https://yteblog.bilgem.tubitak.gov.tr/post-assets/banner/openapi-banner.png

Source: yteblog.bilgem.tubitak.gov.tr



https://i.sstatic.net/nzT6e.png

Source: softwareengineering.stackexchange.com